

Clase 7

TESTING Y DEPURACIÓN

CC1002 Introducción a la
Programación

Procedimientos

- Son “funciones” que no devuelven un valor
 - En el contrato se escribe que devuelven `None`
- Sirven para ejecutar un bloque de instrucciones definido
- Pueden tener instrucciones de interacción (`input`, `print`)
- Los procedimientos pueden ser recursivos

CC1002 Introducción a la
Programación

Procedimientos

```
#saludo: None -> None
#Pregunta por un nombre y
#muestra en pantalla un saludo
def saludo():
    nombre = input('Nombre? ')
    print('Hola ' + nombre)

#temporizador: int -> None
#Muestra en pantalla los numeros
#desde n hasta 1, decrementando en 1
def temporizador(n):
    if n > 0:
        print(n)
        temporizador(n - 1)

#numeros: int -> None
#Muestra en pantalla los numeros
#desde i hasta j, incrementando en 1
def numeros(i, j):
    if i > j:
        return #termina el procedimiento
    else:
        print(i)
        numeros(i + 1, j)
```

CC1002 Introducción a la
Programación

Conceptos

- Testing: acciones que permiten probar la funcionalidad de alguna parte de programa
- Depuración: es el proceso de detectar y corregir errores en un programa
 - Errores de sintaxis, de nombre, de tipo, etc.
 - Errores lógicos (proceso de testing)

CC1002 Introducción a la
Programación

Testing de funciones

- Aspectos generales:
 - La función debe cumplir con el contrato estipulado en la receta de diseño
 - Parámetros
 - Tipo de dato de retorno
 - Los tests de la función deben abarcar suficientes casos para verificar que la función está correctamente programada
 - Objetivo: encontrar errores lógicos

CC1002 Introducción a la
Programación

Testing de funciones

- Aspectos generales:
 - Buen testing para funciones condicionales debe incluir tests para:
 - Todos los casos de borde
 - Al menos un test por cada caso representativo

CC1002 Introducción a la
Programación

Assertions

- Instrucción `assert` sirve para implementar tests en Python
- Sintaxis:

```
assert condicion
```

- `condicion` es una expresión lógica
 - Simple
 - Compuesta, utilizando conectores lógicos

CC1002 Introducción a la
Programación

Assertions

- Ejemplos:

```
assert 10 < 12
assert a == b and (c < d or a < b)
# Suponga que funcionBooleana(x) debe devolver True
# y funcionBooleana(y) debe devolver False
assert funcionBooleana(x) # no agregar: == True
assert not funcionBooleana(y)
# no usar: assert funcionBooleana(y) == False
```

- Condición verdadera: `assert` no hace nada
- Condición es falsa: `assert` arroja un error (`AssertionError`) y el programa termina

CC1002 Introducción a la
Programación

Assertions

- Hasta ahora, hemos usado *assertions* para comprobar que las funciones devuelven el valor correcto para un input determinado
 - Muy importante: implementar el test ANTES que la función
- En general, se pueden utilizar *assertions* en cualquier parte del código
- Se puede usar cualquier operador lógico

CC1002 Introducción a la
Programación

Assertions

- Ejemplo:

```
import random

# escogeAlAzar: None -> int
# Devuelve un numero al azar entre 1 y 10
def escogeAlAzar():
    # random.random() retorna un numero al azar entre 0 y 1
    return int(10 * random.random()) + 1

# Test
valor = escogeAlAzar()
assert 1 <= valor and valor <= 10
```

CC1002 Introducción a la
Programación

Verificar el tipo de dato

- Función `type`

```
>>> valor = 1
>>> assert type(valor) == int
>>> assert type(valor) == bool
Traceback (most recent call last):
  File "<pyshell#6>", line 1, in <module>
    assert type(valor) == bool
AssertionError
```

CC1002 Introducción a la
Programación

Testing de números reales

- No se utiliza `==`, sino la función `cerca`:

```
# cerca: num num num -> bool
# retorna True si x es igual a y con precision epsilon
def cerca(x, y, epsilon):
    diff = x - y
    return abs(diff) < epsilon

# Ejemplo de uso

# Test incorrecto (podria arrojar AssertionError)
# assert 0.1 + 0.2 == 0.3

# Test correcto
tolerancia = 0.000001
assert cerca(0.1 + 0.2, 0.3, tolerancia)
```

CC1002 Introducción a la
Programación

Cálculo de la raíz cuadrada

- Algoritmo (método de Herón):
 - Si z es una estimación de $\sqrt{x}$, pero mayor que $\sqrt{x}$, x/z sería una estimación menor que $\sqrt{x}$
 - Si z es una estimación de $\sqrt{x}$, pero menor que $\sqrt{x}$, x/z sería una estimación mayor que $\sqrt{x}$
 - En ambos casos, promedio entre z y x/z es una mejor estimación de $\sqrt{x}$
 - El proceso se repite hasta alcanzar una precisión suficiente

CC1002 Introducción a la
Programación

Cálculo de la raíz cuadrada

```
# heronRec: num num num -> num
# funcion recursiva que implementa metodo de heron,
# calcula la raiz cuadrada de x
# con precision epsilon y valor inicial de estimacion
# ejemplo : heronRec(2, 0.00001, 1) devuelve 1.414215...
def heronRec(x, epsilon, estimacion):
    # test de pre-condicion
    assert x > 0
    if cerca(estimacion * estimacion, x, epsilon):
        # caso base
        return estimacion
    else :
        # caso recursivo
        mejor_estimacion = (estimacion + (x / estimacion)) / 2
        return heronRec(x, epsilon, mejor_estimacion)
```

CC1002 Introducción a la
Programación

Cálculo de la raíz cuadrada

```
# heron: num num -> num
# wrapper de funcion heronRec
# calcula la raiz cuadrada de x, con precision epsilon
# ejemplo:
# heron(2, 0.1) devuelve 1.416...
# heron(2, 0.00001) devuelve 1.414215...
def heron(x, epsilon):
    # test de tipos de datos
    assert type(x) == int or type(x) == float
    assert type(epsilon) == int or type(epsilon) == float
    # invocar funcion heronRec
    return heronRec(x, epsilon, x / 2)

# Test
import math
from cerca import cerca
assert cerca(heron(2, 0.1), math.sqrt(2), 0.1)
assert cerca(heron(3, 0.01), math.sqrt(3), 0.01)
assert cerca(heron(4, 0.001), math.sqrt(4), 0.001)
```

CC1002 Introducción a la
Programación

Para la próxima clase

- Repasaremos los contenidos de la Unidad I del curso

CC1002 Introducción a la
Programación